CS Fundamentals

Добро пожаловать в руководство по фундаментальным основам Computer Science.

Содержание

- **Алгоритмы** — основы алгоритмов, сложность, сортировки, поиск
- **Структуры данных** — массивы, списки, деревья, графы, хэш-таблицы
- **Архитектура ЭВМ** — устройство компьютера, память, процессор, ввод-вывод
- **Дискретная математика** — логика, множества, комбинаторика, теория графов

Эти знания остаются актуальными десятилетиями и являются основой для понимания любых современных технологий.

Дискретная математика

Дискретная математика — это фундамент всей информатики. В отличие от непрерывной математики (матанализа), она работает с дискретными, отдельными объектами: целыми числами, графами, логическими высказываниями. Именно на ней строятся алгоритмы, базы данных, криптография и компиляторы.

---

1. Математическая логика 🧠

Логика — это язык, на котором формулируются утверждения и доказательства в CS.

Булева алгебра

Основана на двух значениях: **Истина (1/True)** и **Ложь (0/False)**.

Базовые операции:

Операция	Обозначение	Описание	Таблица истинности (A, B → Результат)
НЕ (Отрицание)	`¬A`, `!A`, `NOT A`	Инвертирует значение	0→1, 1→0
И (Конъюнкция)	`A ∧ B`, `A & B`, `A AND B`	Истина, если оба истинны	0,0→0; 0,1→0; 1,0→0; **1,1→1**
ИЛИ (Дизъюнкция)	`A ∨ B`, `A | B`, `A OR B`	Истина, если хотя бы один истинен	0,0→0; **0,1→1**; **1,0→1**; **1,1→1**
Исключающее ИЛИ	`A ⊕ B`, `A XOR B`	Истина, если значения различны	0,0→0; **0,1→1**; **1,0→1**; 1,1→0

Законы булевой алгебры:

- **Коммутативность:** `A ∧ B = B ∧ A`, `A ∨ B = B ∨ A`
- **Ассоциативность:** `(A ∧ B) ∧ C = A ∧ (B ∧ C)`
- **Дистрибутивность:** `A ∧ (B ∨ C) = (A ∧ B) ∨ (A ∧ C)`
- **Законы де Моргана:**
- `¬(A ∧ B) = ¬A ∨ ¬B`
- `¬(A ∨ B) = ¬A ∧ ¬B`

> *Применение:* Упрощение условий в коде, оптимизация запросов, проектирование логических схем.

Предикаты и кванторы

- **Предикат** — функция, возвращающая True/False в зависимости от аргумента: `P(x) = "x > 5"`.
- **Квантор всеобщности (∀):** "Для всех x верно P(x)".
- **Квантор существования (∃):** "Существует x, для которого верно P(x)".

---

2. Теория множеств 📦

Множество — неупорядоченная коллекция уникальных элементов. Основа реляционных баз данных (SQL) и теории типов.

Базовые понятия

- **Обозначение:** `A = {1, 2, 3}`, `B = {2, 4, 6}`
- **Принадлежность:** `2 ∈ A` (2 принадлежит A), `5 ∉ A` (5 не принадлежит)
- **Мощность множества:** `|A| = 3` (количество элементов)
- **Пустое множество:** `∅` или `{}`

Операции над множествами

Операция	Обозначение	Описание	Пример (A={1,2,3}, B={2,4})
Объединение	`A ∪ B`	Все элементы из A и B	`{1, 2, 3, 4}`
Пересечение	`A ∩ B`	Только общие элементы	`{2}`
Разность	`A \ B`	Элементы A, которых нет в B	`{1, 3}`
Симметрическая разность	`A △ B`	Элементы, которые есть только в одном из множеств	`{1, 3, 4}`
Декартово произведение	`A × B`	Все возможные пары (a, b)	`{(1,2), (1,4), (2,2), (2,4), (3,2), (3,4)}`

Подмножества

- **Подмножество (⊆):** `A ⊆ B`, если каждый элемент A содержится в B.
- **Собственное подмножество (⊂):** `A ⊂ B`, если `A ⊆ B` и `A ≠ B`.
- **Булеан (множество всех подмножеств):** `P(A)`. Если `|A| = n`, то `|P(A)| = 2^n`.

> *Применение в IT:* SQL (`JOIN`, `UNION`, `INTERSECT`), проверка прав доступа (роли как множества), теория типов в языках программирования.

---

3. Комбинаторика 🔢

Наука о подсчете количества вариантов. Критична для анализа сложности алгоритмов и криптографии.

Основные правила

1. **Правило суммы:** Если объект A можно выбрать `m` способами, а объект B — `n` способами, и выборы не пересекаются, то выбор "A или B" возможен `m + n` способами.

2. **Правило произведения:** Если объект A можно выбрать `m` способами, и после каждого такого выбора объект B можно выбрать `n` способами, то пару (A, B) можно выбрать `m × n` способами.

Формулы

Концепция	Формула	Описание	Пример
Перестановки (без повторений)	`P_n = n!`	Сколькими способами можно упорядочить n элементов	3 книги на полке: `3! = 6` способов
Перестановки с повторениями	`P = n! / (n₁! × n₂! × ...)`	Если есть одинаковые элементы	Слово "ООК": `3! / 2! = 3` способа (ООК, ООО, КОО)
Размещения (порядок важен)	`A_n^k = n! / (n-k)!`	Выбор k элементов из n с учетом порядка	3 призовых места из 10 участников: `10! / 7! = 720`
Сочетания (порядок НЕ важен)	`C_n^k = n! / (k! × (n-k)!)`	Выбор k элементов из n без учета порядка	Выбор 3 дежурных из 10: `C_10^3 = 120`

Биномиальные коэффициенты:

Формула бинома Ньютона: `(a + b)^n = Σ C_n^k × a^(n-k) × b^k`

> *Применение:* Подсчет вариантов паролей, анализ worst-case сценариев алгоритмов, вероятность коллизий в хэш-таблицах.

---

4. Теория графов 🕸

Граф — структура из вершин (узлов) и соединяющих их ребер. Моделирует сети, социальные связи, маршруты, зависимости.

Основные понятия

- **Вершина (узел):** Объект графа.
- **Ребро (дуга):** Связь между вершинами.
- **Неориентированный граф:** Ребра двусторонние (A-B).
- **Ориентированный граф (Digraph):** Ребра имеют направление (A→B).
- **Степень вершины:** Количество инцидентных ей ребер.
- **Путь:** Последовательность вершин, соединенных ребрами.
- **Цикл:** Путь, начинающийся и заканчивающийся в одной вершине.

Виды графов

Тип	Описание	Пример применения
Связный	Между любыми двумя вершинами есть путь	Социальная сеть (все друзья связаны)
Полный	Каждая вершина соединена со всеми остальными	Турнирная таблица (каждый с каждым)
Дерево	Связный граф без циклов	Файловая система, DOM дерева
Взвешенный	Ребрам присвоены числа (вес)	Карта дорог (расстояние/время)

| **Двудольный** | Вершины делятся на 2 множества, ребра только между ними | Задачи соответствия (работники-задачи) |

Алгоритмы на графах (кратко)

- **Поиск в ширину (BFS):** Кратчайший путь в невзвешенном графе. O(V+E).
- **Поиск в глубину (DFS):** Обход, поиск циклов, топологическая сортировка. O(V+E).
- **Алгоритм Дейкстры:** Кратчайший путь во взвешенном графе (без отрицательных весов). O((V+E)logV).
- **Алгоритм Флойда-Уоршелла:** Кратчайшие пути между всеми парами вершин. $O(V^3)$.

> *Применение:* Маршрутизация в интернете (OSPF, BGP), рекомендации друзей, поиск зависимостей в пакетных менеджерах (npm, pip), GPS-навигация.

---

5. Отношения и функции 🔗

Отношения

Отношение R между множествами A и B — подмножество их декартова произведения `A × B`.

Свойства отношений на одном множестве:

- **Рефлексивность:** ∀x: xRx (каждый элемент связан сам с собой). Пример: `≤`.
- **Симметричность:** Если xRy, то yRx. Пример: "быть братом".
- **Транзитивность:** Если xRy и yRz, то xRz. Пример: `<`, `=`, `≤`.
- **Антисимметричность:** Если xRy и yRx, то x=y. Пример: `≤`.

Важные типы отношений:

- **Отношение эквивалентности:** Рефлексивно, симметрично, транзитивно. Разбивает множество на классы эквивалентности. Пример: "иметь одинаковый остаток от деления на 5".
- **Отношение порядка:** Рефлексивно, антисимметрично, транзитивно. Пример: `≤` на числах.

Функции

Функция `f: A → B` ставит в соответствие каждому элементу A ровно один элемент B.

Свойства функций:

Свойство	Определение	Пример
Инъективность (1-к-1)	Разным элементам A соответствуют разные элементы B	`f(x) = 2x` на натуральных числах
Сюръективность (onto)	Каждый элемент B имеет прообраз в A	`f(x) = x mod 3` на Z→{0,1,2}
Биективность	Одновременно инъективна и сюръективна (взаимно однозначное соответствие)	`f(x) = x + 1` на Z→Z

> *Применение:* Хэш-функции (должны минимизировать коллизии → стремиться к инъективности), шифрование (биективные преобразования), нормализация БД.

📝 Шпаргалка: Формулы для запоминания

```
Факториал:          n! = 1 × 2 × ... × n
Перестановки:       P_n = n!
Размещения:         A_n^k = n! / (n-k)!
Сочетания:          C_n^k = n! / (k! × (n-k)!)
Сумма степеней 2:   Σ(2^i, i=0..n) = 2^(n+1) - 1
Число подмножеств:  |P(A)| = 2^|A|
Число ребер в полном графе: E = n(n-1)/2
```

🎯 Практическое применение в разработке

1. **Базы данных:** Реляционная алгебра основана на теории множеств. JOIN — это декартово произведение с фильтрацией.
2. **Поисковые системы:** Булева логика для запросов ("коты И собаки НЕ птицы").
3. **Сети:** Графы для моделирования топологии, маршрутизации.
4. **Криптография:** Теория чисел (раздел дискретной математики) для RSA, эллиптических кривых.
5. **Машинное обучение:** Теория вероятностей (базируется на мере и множествах) для байесовских моделей.
6. **Компиляторы:** Графы зависимостей для оптимизации кода, конечные автоматы (раздел теории графов) для лексического анализа.

> **Запомните:** Без понимания дискретной математики вы будете просто копировать код из StackOverflow. С её пониманием — вы сможете создавать новые алгоритмы и решать задачи, которых ещё не было в интернете.

Архитектура компьютера

Архитектура компьютера — это описание структуры и принципов работы вычислительной системы. Понимание того, как работает железо "под капотом", делает вас лучшим программистом: вы пишете более эффективный код, понимаете узкие места и можете оптимизировать производительность.

---

1. Основные компоненты компьютера 🖥️

Процессор (CPU — Central Processing Unit)

"Мозг" компьютера, выполняющий инструкции программы.

Структура CPU:

Компонент	Назначение	Пример
АЛУ (ALU)	Арифметико-логические операции: сложение, вычитание, сравнение	`5 + 3 = 8`, `x > y`
УУ (CU)	Декодирование инструкций, управление потоком данных	Чтение команды из памяти, отправка сигналов в АЛУ

| **Регистры** | Сверхбыстрая память внутри CPU для временного хранения данных | `EAX`, `EBX` (x86), `R0-R15` (ARM) |

| **Кэш L1/L2/L3** | Быстрая буферная память между CPU и RAM | L1: 32-64 KB на ядро, L3: несколько MB на весь CPU |

Тактовая частота: Измеряется в ГГц (миллиардах циклов в секунду). За один такт может выполняться одна или несколько микроопераций.

> **Важно:** Больше ГГц ≠ всегда быстрее. Архитектура (IPC — instructions per cycle) важнее чистой частоты.

Память (Memory Hierarchy) 📊

Иерархия памяти строится по принципу **"быстрее = меньше = дороже"**:

```
┌──────────────────────────────────┐
│          Регистры CPU            │ ← 1 цикл, ~1 байт
│          (самая быстрая)         │
├──────────────────────────────────┤
│          Кэш L1 (на ядро)        │ ← 3-4 цикла, ~32-64 KB
├──────────────────────────────────┤
│          Кэш L2 (на ядро)        │ ← 10-12 циклов, ~256 KB
├──────────────────────────────────┤
│          Кэш L3 (общий)          │ ← 30-40 циклов, ~8-32 MB
├──────────────────────────────────┤
│    Оперативная память (RAM)      │ ← 100+ циклов, ~8-64 GB
├──────────────────────────────────┤
│    SSD / NVMe (вторичная память) │ ← 100,000+ циклов, ~256 GB - 4 TB
├──────────────────────────────────┤
│    HDD (медленная вторичная)     │ ← 1,000,000+ циклов, ~1-10 TB
└──────────────────────────────────┘
```

Типы памяти:

- **RAM (Random Access Memory):** Энергозависимая, быстрая, для работающих программ.
- **DRAM:** Дешевле, медленнее, используется для основной памяти.
- **SRAM:** Быстрее, дороже, используется для кэша.
- **ROM (Read-Only Memory):** Энергонезависимая, хранит прошивки (BIOS/UEFI).
- **Flash (SSD, USB):** Энергонезависимая, для долговременного хранения.

Ввод-вывод (I/O) 🔌

Способ взаимодействия CPU с периферийными устройствами (клавиатура, диск, сеть).

Механизмы I/O:

1. **Программируемый I/O:** CPU активно опрашивает устройство (неэффективно).
2. **Прерывания (Interrupts):** Устройство сигнализирует CPU о готовности данных. CPU приостанавливает текущую задачу, обрабатывает прерывание, возвращается обратно.
3. **DMA (Direct Memory Access):** Специальный контроллер передает данные между устройством и памятью **без участия CPU**. CPU только инициирует передачу и получает уведомление о завершении.

> *Пример:* Загрузка файла с диска в память. DMA освобождает CPU для других задач.

---

2. Принципы работы процессора ⚙️

Фон-неймановская архитектура

Классическая архитектура, предложенная Джоном фон Нейманом (1945):

Основные принципы:

1. **Единая память:** Программы и данные хранятся в одной памяти.
2. **Последовательное выполнение:** Команды выполняются одна за другой.
3. **Адресуемость памяти:** Каждая ячейка памяти имеет уникальный адрес.
4. **Условные переходы:** Возможность изменять порядок выполнения команд (if, loops).

Цикл выполнения команды (Fetch-Decode-Execute):

```
1. Fetch (Выборка):    Получить команду из памяти по адресу PC (Program Counter)
2. Decode (Декодирование): Определить, что нужно сделать
3. Execute (Выполнение): Выполнить операцию (АЛУ, запись в память, переход)
4. Update PC:          Увеличить счетчик команд (или установить новый адрес при переходе)
```

> **Проблема фон-неймановского бутылочного горлышка:** Пропускная способность шины между CPU и памятью ограничивает скорость работы. Решение → кэш-память.

Конвейеризация (Pipelining) 🏭

Техника ускорения выполнения команд за счет параллелизма на уровне стадий.

Идея: Разбить выполнение команды на стадии и выполнять разные команды на разных стадиях одновременно.

Классический 5-ступенчатый конвейер (MIPS):

```
Время →
Команда 1: IF → ID → EX → MEM → WB
Команда 2:      IF → ID → EX → MEM → WB
Команда 3:           IF → ID → EX → MEM → WB
Команда 4:                IF → ID → EX → MEM → WB
```

- **IF (Instruction Fetch):** Выборка команды
- **ID (Instruction Decode):** Декодирование и чтение регистров
- **EX (Execute):** Выполнение операции
- **MEM (Memory Access):** Доступ к памяти
- **WB (Write Back):** Запись результата в регистр

Преимущество: В идеале — **1 команда за такт** (vs 5 тактов на команду без конвейера).

Проблемы конвейера:

- **Конфликты данных:** Команда зависит от результата предыдущей, которая ещё не завершилась.
- *Решение:* Forwarding (пересылка результатов напрямую), stall (остановка конвейера).
- **Конфликты управления:** Условный переход (if/else) — неизвестно, какую команду грузить дальше.
- *Решение:* Предсказание переходов (branch prediction), спекулятивное выполнение.

Многозадачность и многопоточность 🔄

Многозадачность (Multitasking): Быстрое переключение между задачами (процессами), создающее иллюзию параллельной работы.

Переключение контекста:

1. Сохранить состояние текущего процесса (регистры, PC, стек) в PCB (Process Control Block).
2. Загрузить состояние следующего процесса из его PCB.
3. Продолжить выполнение.

> **Цена переключения:** ~1-10 мкс. Частые переключения снижают производительность.

Многопоточность (Multithreading):

- **Аппаратная (Hyper-Threading):** Одно физическое ядро выполняет несколько потоков, переключаясь между ними при ожиданиях (например, доступ к памяти).
- **Программная:** ОС планирует потоки одного или разных процессов на разных ядрах.

---

3. Представление данных 💾

Числа в компьютере

Двоичная система (base-2):

- Основание: 2 (цифры 0, 1).
- Пример: `1011₂ = 1×8 + 0×4 + 1×2 + 1×1 = 11₁₀`

Шестнадцатеричная система (base-16):

- Основание: 16 (цифры 0-9, A-F).
- Удобна для записи байтов: `FF₁₆ = 255₁₀ = 11111111₂`.
- Пример адреса памяти: `0x7FFF_FFFF`.

Целые числа:

- **Без знака (unsigned):** Все биты — значение. Диапазон для 8 бит: `0..255`.
- **Со знаком (signed):** Старший бит — знак (0=+, 1=-), остальные — значение в **дополнительном коде (Two's Complement)**.
- Дополнительный код: инвертировать все биты и прибавить 1.
- Пример (-5 в 8 битах): `5 = 00000101` → инверсия `11111010` → +1 = `11111011`.
- Диапазон для 8 бит: `-128..127`.

Числа с плавающей точкой (IEEE 754):

Формат для вещественных чисел: `(-1)^sign × mantissa × 2^(exponent-bias)`

Тип	Биты	Диапазон	Точность
float32	32 (1+8+23)	±10^±38	~7 десятичных цифр
float64	64 (1+11+52)	±10^±308	~15 десятичных цифр

> **Важно:** `0.1 + 0.2 ≠ 0.3` в float из-за неточности представления. Не используйте float для финансовых расчетов!

Кодировки текста

| Кодировка | Описание | Размер символа |

|----------|----------|----------|

| **ASCII** | 7 бит, 128 символов (латиница, цифры, управляющие) | 1 байт |

| **Latin-1 (ISO-8859-1)** | 8 бит, 256 символов (ASCII + европейские буквы) | 1 байт |

| **UTF-8** | Переменная длина (1-4 байта), обратно совместима с ASCII | 1-4 байта |

| **UTF-16** | 2 или 4 байта на символ, используется в Windows/Java | 2-4 байта |

UTF-8 (стандарт интернета):

- ASCII (0-127): 1 байт (`0xxxxxxx`).
- Кириллица (128-2047): 2 байта (`110xxxxx 10xxxxxx`).
- Китайские иероглифы: 3 байта.
- Эмодзи: 4 байта.

> **Проблема:** "Кракозябры" возникают при неправильном определении кодировки. Всегда явно указывайте `encoding='utf-8'` при работе с файлами.

---

4. Оптимизация кода с учетом архитектуры 🚀

Локальность данных

Временная локальность: Если к данным обратились сейчас, скорее всего, обратятся снова скоро.

- *Совет:* Повторно используйте переменные, держите "горячие" данные в кэше.

Пространственная локальность: Если обратились к данным, скорее всего, обратятся к соседним.

- *Совет:* Обрабатывайте массивы последовательно, используйте структуры вместо массивов структур (SoA vs AoS).

Пример (плохо vs хорошо):

```
# ❌ Плохо: скачкообразный доступ к памяти (промах кэша)
for i in range(0, 1000000, 1000):
    process(array[i])

# ✅ Хорошо: последовательный доступ (попадание в кэш)
for i in range(1000000):
    process(array[i])
```

Выравнивание данных (Alignment)

CPU обращается к памяти блоками (обычно 4 или 8 байт). Если данные не выровнены по границе блока, требуется 2 обращения вместо 1.

```
// ❌ Плохо: структура 9 байт, выравнивание нарушено
struct Bad {
    char a;     // 1 байт
    int b;      // 4 байта (смещение 1, нужно 3 байта padding)
    short c;    // 2 байта
}; // Итого: 1 + 3(padding) + 4 + 2 = 10 байт (не 7!)

// ✅ Хорошо: явное выравнивание
struct Good {
    int b;      // 4 байта
    short c;    // 2 байта
    char a;     // 1 байт
    // 1 байт padding до границы 4
}; // Итого: 8 байт
```

Branch Prediction (Предсказание переходов)

CPU предсказывает, куда пойдет ветвление (if/else), и начинает выполнять команды заранее. Если предсказание неверно — конвейер сбрасывается (~10-20 тактов потерь).

Совет: Избегайте непредсказуемых условий в горячих циклах.

```
# ❌ Плохо: случайные данные, предсказатель ошибается в 50% случаев
for x in random_array:
    if x > 128:
        sum += x

# ✅ Хорошо: отсортированные данные, предсказатель работает идеально
for x in sorted_array:
    if x > 128:
        sum += x
```

📝 Шпаргалка: Порядок величин

Операция	Время	Эквивалент в циклах CPU (при 3 ГГц)
Такт CPU	0.3 нс	1 цикл
Доступ к кэшу L1	1 нс	~3 цикла
Доступ к кэшу L2	4 нс	~12 циклов
Доступ к кэшу L3	15 нс	~45 циклов
Доступ к RAM	100 нс	~300 циклов
Чтение SSD (NVMe)	100 мкс	~300,000 циклов
Чтение HDD	10 мс	~30,000,000 циклов
Сетевой запрос (RTT)	1-100 мс	~3M - 300M циклов

> **Вывод:** Один промах кэша — это сотни потерянных циклов. Одна операция с диском — миллионы циклов. **Оптимизируйте доступ к памяти!**

🎯 Практическое применение в разработке

1. **Базы данных:** B-деревья спроектированы с учетом размера страницы диска (4 KB) и кэш-линий CPU.
2. **Игры:** Data-oriented design для максимальной утилизации кэша и SIMD-инструкций.
3. **Веб-серверы:** Асинхронный I/O (epoll, kqueue) вместо потоков для обработки тысяч соединений без переключения контекста.
4. **Big Data:** Columnar storage (Parquet, ORC) для лучшей компрессии и локальности при аналитических запросах.
5. **Криптография:** Защита от timing attacks через постоянное время выполнения операций.

> **Запомните:** Хороший программист знает синтаксис языка. Отличный программист понимает, как его код выполняется на железе.

Структуры данных

Структуры данных — это способы организации и хранения информации для эффективного доступа и модификации. Выбор правильной структуры данных может ускорить программу в сотни раз. Это знание **не зависит от языка** — массивы в C++, Python, Java или Go работают по одним и тем же принципам.

---

1. Линейные структуры 📏

Массивы (Arrays)

Определение: Непрерывный блок памяти, хранящий элементы одного типа.

Характеристики:

Операция	Сложность	Описание
Доступ по индексу	**O(1)**	Мгновенно: `address = base + index * element_size`
Поиск	O(n)	Нужно перебрать все элементы
Вставка в конец	O(1)*	Амортизированно (если есть место)
Вставка в середину	**O(n)**	Нужно сдвинуть все элементы справа
Удаление	**O(n)**	Нужно сдвинуть элементы и заполнить дыру

Плюсы:

- ✅ Быстрый доступ по индексу
- ✅ Компактное хранение (нет накладных расходов на указатели)
- ✅ Кэш-локальность (элементы лежат рядом в памяти)

Минусы:

- ❌ Фиксированный размер (в статических массивах)
- ❌ Дорогая вставка/удаление в середине

Пример использования: Хранение пикселей изображения, буфер аудио, матрицы.

```
# Python: списки — динамические массивы
arr = [10, 20, 30, 40, 50]
print(arr[2])      # O(1): 30
arr.insert(2, 25)  # O(n): сдвиг элементов [10, 20, 25, 30, 40, 50]
```

Связные списки (Linked Lists)

Определение: Элементы (узлы) хранятся в произвольных местах памяти, каждый содержит данные и указатель на следующий узел.

Виды:

- **Односвязный:** Каждый узел → следующий.
- **Двусвязный:** Узел → следующий + предыдущий.
- **Циклический:** Последний узел → первый.

Характеристики:

Операция	Сложность	Описание
Доступ по индексу	**O(n)**	Нужно пройти от головы до элемента
Поиск	O(n)	Перебор всех узлов
Вставка в начало	**O(1)**	Изменить указатель головы
Вставка в середину	O(1)*	Если известен узел перед точкой вставки
Удаление	O(1)*	Если известен узел

Плюсы:

- ✅ Динамический размер (растет/сжимается при необходимости)
- ✅ Быстрая вставка/удаление (без сдвига элементов)
- ✅ Нет фрагментации памяти

Минусы:

- ❌ Медленный доступ по индексу
- ❌ Дополнительные расходы на указатели (2x-3x больше памяти)
- ❌ Плохая кэш-локальность (узлы разбросаны в памяти)

Пример использования: Реализация стеков, очередей, undo/redo в редакторах.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Вставка в начало: O(1)
new_node = Node(100)
new_node.next = head
head = new_node
```

Стеки (Stacks)

Определение: Структура LIFO (Last In, First Out) — последний пришел, первый ушел.

Операции:

- `push(x)` — добавить элемент на вершину. O(1).
- `pop()` — удалить и вернуть верхний элемент. O(1).
- `peek()` — посмотреть верхний элемент без удаления. O(1).

Применение:

- Вызов функций (стек вызовов).
- Отмена действий (Ctrl+Z).
- Проверка скобочных последовательностей.
- DFS (поиск в глубину).

```
stack = []
stack.append(1)   # push
stack.append(2)
top = stack.pop() # pop → 2
```

Задача: Проверка правильности скобочной последовательности.

```
def is_valid(s: str) -> bool:
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}
    for char in s:
        if char in mapping:
            top = stack.pop() if stack else '#'
            if mapping[char] != top:
                return False
        else:
            stack.append(char)
    return not stack
```

Очереди (Queues)

Определение: Структура FIFO (First In, First Out) — первый пришел, первый ушел.

Операции:

- `enqueue(x)` — добавить в хвост. O(1).
- `dequeue()` — удалить из головы. O(1).
- `front()` — посмотреть первый элемент. O(1).

Варианты:

- **Обычная очередь:** FIFO.
- **Дек (Deque):** Двусторонняя очередь, вставка/удаление с обоих концов.
- **Приоритетная очередь:** Элементы извлекаются по приоритету (реализуется через кучу).

Применение:

- Обработка запросов к серверу.
- Печать документов.
- BFS (поиск в ширину).
- Буферизация потоков данных.

```
from collections import deque
queue = deque()
queue.append(1)      # enqueue
queue.append(2)
first = queue.popleft()  # dequeue → 1
```

---

2. Иерархические структуры 🌳

Деревья (Trees)

Определение: Иерархическая структура из узлов (вершин), соединенных ребрами. Один корневой узел, каждый узел имеет 0 или более дочерних.

Термины:

- **Корень (Root):** Верхний узел без родителя.
- **Лист (Leaf):** Узел без детей.
- **Высота:** Максимальное количество ребер от корня до листа.
- **Глубина:** Количество ребер от корня до данного узла.

Бинарное дерево поиска (BST — Binary Search Tree)

Свойство: Для каждого узла:

- Все значения в левом поддереве < значения узла.
- Все значения в правом поддереве > значения узла.

Характеристики:

Операция	Сложность (средняя)	Сложность (худшая)
Поиск	O(log n)	**O(n)** (вырожденное дерево)
Вставка	O(log n)	O(n)
Удаление	O(log n)	O(n)

Проблема: При неудачном порядке вставки дерево вырождается в связный список.

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def search(root, target):
    if not root or root.val == target:
        return root
    if target < root.val:
        return search(root.left, target)
    return search(root.right, target)
```

Сбалансированные деревья

Решают проблему вырождения BST, поддерживая высоту O(log n).

Тип	Принцип балансировки	Применение
AVL-дерево	Разница высот поддеревьев ≤ 1	Там, где много поисков (словари)
Красно-черное дерево	Правила окраски узлов (красный/черный)	STL (C++), TreeMap (Java), ассоциативные массивы
B-дерево	Множественные дети на узел (для дисков)	Базы данных, файловые системы
Дерево отрезков	Хранение агрегатов на отрезках	Запросы диапазона (сумма, мин, макс)

Применение деревьев:

- Индексы в базах данных (B-деревья).
- Автодополнение (Trie, префиксное дерево).
- Сжатие данных (Huffman tree).
- Маршрутизация в сетях (routing tables).

---

3. Хэш-таблицы (Hash Tables) ⚡

Определение: Структура, отображающая ключи в значения через хэш-функцию. Обеспечивает быстрый поиск, вставку и удаление.

Принцип работы:

1. Вычисляется хэш ключа: `index = hash(key) % capacity`.
2. Значение сохраняется в ячейку `table[index]`.

Характеристики:

Операция	Сложность (средняя)	Сложность (худшая)
Поиск	**O(1)**	O(n) (много коллизий)
Вставка	**O(1)**	O(n)
Удаление	**O(1)**	O(n)

Коллизии: Ситуация, когда два ключа дают одинаковый хэш.

Методы разрешения коллизий:

1. **Метод цепочек:** В каждой ячейке — связный список (или дерево) элементов.
2. **Открытая адресация:** При коллизии ищем следующую свободную ячейку (линейный пробег, квадратичный, двойное хэширование).

Требования к хэш-функции:

- Детерминированность: `hash(x)` всегда одинаков для одного x.
- Равномерность: Хэши распределяются равномерно по таблице.
- Эффективность: Быстрое вычисление.

Пример (Python dict):

```
d = {'name': 'Alice', 'age': 25, 'city': 'Moscow'}
print(d['name'])  # O(1): Alice
d['age'] = 26     # O(1): обновление
```

Применение:

- Словари, ассоциативные массивы.
- Кэширование (memoization).
- Подсчет частоты элементов.
- Проверка дубликатов.
- Реализация множеств (set).

---

4. Продвинутые структуры 🚀

Куча (Heap)

Определение: Бинарное дерево, удовлетворяющее свойству кучи:

- **Max-heap:** Каждый узел ≥ своих детей (корень — максимум).
- **Min-heap:** Каждый узел ≤ своих детей (корень — минимум).

Реализация: Обычно массивом. Для узла `i`:

- Левый ребенок: `2*i + 1`
- Правый ребенок: `2*i + 2`
- Родитель: `(i-1) // 2`

Характеристики:

Операция	Сложность
Вставка	O(log n)
Извлечение мин/макс	O(log n)
Получение мин/макс	**O(1)**

Применение:

- Приоритетные очереди.
- Алгоритм Дейкстры (кратчайший путь).
- Сортировка Heap Sort.
- Поиск K наибольших/наименьших элементов.

```
import heapq
heap = []
heapq.heappush(heap, 10)
heapq.heappush(heap, 5)
heapq.heappush(heap, 20)
min_val = heapq.heappop(heap)  # 5
```

Trie (Префиксное дерево)

Определение: Дерево для хранения строк, где каждый узел — символ, а путь от корня до узла — строка.

Применение:

- Автодополнение (Google Search).
- Проверка орфографии.

- IP-маршрутизация (longest prefix match).
- Словари для игр (Scrabble).

```
      (root)
     /   |   \
    c    a    t
   / \   |    |
  a   h  r    e
 /     \|    |
t      t p   a
|           |
s           r
```

Слова: cat, cats, chat, art, car, tea, tear

Skip List (Список с пропусками)

Определение: probabilistic структура, альтернатива сбалансированным деревьям. Многоуровневый связный список с "экспресс-полосами".

Сложность: O(log n) для поиска, вставки, удаления.

Преимущества перед AVL/Red-Black:
- Проще в реализации.
- Легче распараллеливается.
- Используется в Redis, LevelDB.

📊 Сравнительная таблица структур данных

Структура	Доступ	Поиск	Вставка	Удаление	Память	Лучшее применение
Массив	O(1)	O(n)	O(n)	O(n)	Низкая	Индексный доступ, кэш-френдли
Связный список	O(n)	O(n)	O(1)	*O(1)*	Средняя	Частые вставки/удаления
Стек	O(n)	O(n)	O(1)	O(1)	Низкая	LIFO, undo, DFS
Очередь	O(n)	O(n)	O(1)	O(1)	Низкая	FIFO, буферы, BFS
BST	O(log n)	O(log n)	O(log n)	O(log n)	Средняя	Упорядоченные данные
AVL / Red-Black	O(log n)	O(log n)	O(log n)	O(log n)	Высокая	Гарантированный log n
Хэш-таблица	N/A	O(1)	O(1)	O(1)	Высокая	Быстрый поиск по ключу
Куча	N/A	O(n)	O(log n)	O(log n)	Низкая	Приоритеты, min/max
Trie	O(m)	O(m)	O(m)	O(m)	Очень высокая	Строки, префиксы

*m = длина ключа/строки

🎯 Как выбрать структуру данных?

Задайте себе вопросы:

1. **Нужен ли быстрый доступ по индексу?** → Массив.
2. **Частые вставки/удаления в середине?** → Связный список.
3. **Нужен порядок LIFO или FIFO?** → Стек или очередь.
4. **Быстрый поиск по ключу?** → Хэш-таблица.
5. **Нужно хранить данные отсортированными?** → BST / AVL / Red-Black.
6. **Работа со строками и префиксами?** → Trie.
7. **Нужен приоритетный доступ (min/max)?** → Куча.
8. **Ограничения по памяти?** → Избегайте структур с указателями (списки, деревья).

> **Золотое правило:** Нет "лучшей" структуры данных. Есть та, которая лучше подходит для **вашей** задачи.

💡 Практическое применение

1. **Базы данных:** B-деревья для индексов, хэш-таблицы для JOIN.
2. **Компиляторы:** Символьные таблицы (хэш-таблицы), AST (деревья).
3. **Сети:** Таблицы маршрутизации (Trie, хэш-таблицы).
4. **OS:** Планировщик процессов (приоритетная очередь), управление памятью (списки свободных блоков).
5. **Поисковики:** Инвертированные индексы (хэш-таблицы + списки), автодополнение (Trie).
6. **Кэши:** LRU cache (хэш-таблица + двусвязный список).

> **Запомните:** Глубокое понимание структур данных отличает junior от senior разработчика. Вы не будете изобретать велосипед, но сможете выбрать правильный инструмент для задачи.

Алгоритмы

Алгоритмы — это сердце Computer Science. Это пошаговые инструкции для решения задач. Понимание алгоритмов позволяет выбирать оптимальные решения, оценивать их эффективность и создавать новые алгоритмы для уникальных задач. Как и структуры данных, алгоритмы **не зависят от языка программирования**.

---

1. Оценка сложности алгоритмов (Big O Notation) 📈

Big O описывает, как время выполнения или потребление памяти растет с увеличением размера входных данных (n).

Основные классы сложности

Сложность	Название	Пример	n=10	n=1000	n=1,000,000
O(1)	Константное	Доступ к элементу массива	1	1	1
O(log n)	Логарифмическое	Бинарный поиск	~3	~10	~20
O(n)	Линейное	Поиск в массиве	10	1000	1,000,000
O(n log n)	Линейно-логарифмическое	Быстрая сортировка	~30	~10,000	~20,000,000

O(n²)	Квадратичное	Сортировка пузырьком	100	1,000,000	1,000,000,000,000
O(2ⁿ)	Экспоненциальное	Перебор всех подмножеств	1024	❌	❌
O(n!)	Факториальное	Задача коммивояжера (перебор)	3,628,800	❌	❌

> **Важно:** Big O описывает **худший случай** (upper bound). На практике алгоритм может работать быстрее.

Правила упрощения Big O

1. **Отбрасываем константы:** `O(2n) = O(n)`, `O(n/2) = O(n)`
2. **Отбрасываем младшие члены:** `O(n² + n) = O(n²)`, `O(n³ + n² + n) = O(n³)`
3. **Сумма сложностей:** Если выполняются последовательно → складываем: `O(n) + O(n²) = O(n²)`
4. **Произведение сложностей:** Если вложены друг в друга → умножаем: `O(n) × O(n) = O(n²)`

Примеры анализа

```
# O(1) - Константное
def get_first(arr):
    return arr[0]

# O(n) - Линейное
def find_max(arr):
    max_val = arr[0]
    for x in arr:      # n итераций
        if x > max_val:
            max_val = x
    return max_val

# O(n²) - Квадратичное
def print_pairs(arr):
    for i in arr:          # n итераций
        for j in arr:      # n итераций
            print(i, j)    # Всего: n × n = n² операций

# O(log n) - Логарифмическое
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1   # Отбрасываем половину
        else:
            right = mid - 1  # Отбрасываем половину
    return -1
```

---

2. Алгоритмы сортировки 🔀

Сравнительная таблица

Алгоритм	Лучший	Средний	Худший	Память	Стабильный	Применение

Пузырьковая	O(n)	O(n²)	O(n²)	O(1)	✅ Да	Только для обучения
Выбором	O(n²)	O(n²)	O(n²)	O(1)	❌ Нет	Только для обучения
Вставками	O(n)	O(n²)	O(n²)	O(1)	✅ Да	Малые массивы (<50)
Быстрая (Quick)	O(n log n)	O(n log n)	O(n²)	O(log n)	❌ Нет	**Универсальная**
Слиянием (Merge)	O(n log n)	O(n log n)	O(n log n)	O(n)	✅ Да	Linked lists, внешняя сортировка
Пирамидальная (Heap)	O(n log n)	O(n log n)	O(n log n)	O(1)	❌ Нет	Когда мало памяти
Timsort	O(n)	O(n log n)	O(n log n)	O(n)	✅ Да	Python `.sort()`, Java `.sort()`

Пузырьковая сортировка (Bubble Sort)

Идея: Многократно проходим по массиву, меняя местами соседние элементы, если они в неправильном порядке.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:  # Оптимизация: если не было обменов, массив уже отсортирован
            break
```

- **Плюсы:** Простота реализации.
- **Минусы:** Очень медленно O(n²).
- **Применение:** Только в учебных целях.

Сортировка вставками (Insertion Sort)

Идея: Берем элемент и вставляем его на правильное место среди уже отсортированных элементов слева.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```

- **Плюсы:** Эффективен на почти отсортированных данных O(n), стабилен, сортирует "на месте".
- **Минусы:** O(n²) в среднем случае.
- **Применение:** Малые массивы (<50 элементов), финальная стадия QuickSort.

Быстрая сортировка (QuickSort)

Идея: Выбираем опорный элемент (pivot), разделяем массив на две части (меньше pivot и больше pivot), рекурсивно сортируем части.

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

Оптимизированная версия (in-place):

```
def quick_sort_inplace(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        pivot_index = partition(arr, low, high)
        quick_sort_inplace(arr, low, pivot_index - 1)
        quick_sort_inplace(arr, pivot_index + 1, high)
    return arr

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

- **Плюсы:** Очень быстро на практике O(n log n), сортирует "на месте".
- **Минусы:** Нестабилен, худший случай O(n^2) (решается рандомизацией pivot).
- **Применение:** Стандартная сортировка во многих языках.

Сортировка слиянием (MergeSort)

Идея: Разделяем массив пополам, рекурсивно сортируем половины, сливаем отсортированные половины.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

- **Плюсы:** Гарантированный O(n log n), стабилен.
- **Минусы:** Требует O(n) дополнительной памяти.
- **Применение:** Сортировка linked lists, внешняя сортировка (когда данные не помещаются в RAM).

Пирамидальная сортировка (HeapSort)

Идея: Строим max-heap из массива, многократно извлекаем максимум (корень) и перестраиваем кучу.

```
def heap_sort(arr):
    n = len(arr)
    # Построение кучи
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    # Извлечение элементов
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]  # Перемещаем корень в конец
        heapify(arr, i, 0)
    return arr

def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

- **Плюсы:** O(n log n) всегда, сортирует "на месте" O(1).
- **Минусы:** Нестабилен, медленнее QuickSort на практике.
- **Применение:** Когда критична память.

---

3. Алгоритмы поиска 🔍

Линейный поиск (Linear Search)

Идея: Последовательно проверяем каждый элемент.

```
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1
```

- **Сложность:** O(n)
- **Применение:** Несохраненные массивы, малые n.

Бинарный поиск (Binary Search)

Идея: Сравниваем с серединой, отбрасываем половину, повторяем. **Только для отсортированных массивов!**

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

- **Сложность:** O(log n)
- **Применение:** Поиск в отсортированных данных, бинарный поиск по ответу.

Поиск в ширину (BFS) и глубину (DFS) на графах

BFS (Queue):

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    while queue:
        vertex = queue.popleft()
        print(vertex)
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

DFS (Stack/Recursion):

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

---

4. Алгоритмы на графах 🕸️

Алгоритм Дейкстры (кратчайший путь)

Идея: Жадный алгоритм для поиска кратчайшего пути во взвешенном графе без отрицательных весов.

```
import heapq

def dijkstra(graph, start):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    pq = [(0, start)]

    while pq:
        current_dist, current_node = heapq.heappop(pq)

        if current_dist > distances[current_node]:
            continue

        for neighbor, weight in graph[current_node].items():
            distance = current_dist + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(pq, (distance, neighbor))

    return distances
```

- **Сложность:** O((V+E) log V)
- **Применение:** GPS-навигация, маршрутизация в сетях.

---

5. Динамическое программирование (DP) 🧩

Идея: Разбиение задачи на перекрывающиеся подзадачи, решение каждой один раз и сохранение результата (memoization/tabulation).

Классические задачи DP

1. Числа Фибоначчи:

```
# ❌ Рекурсия без мемоизации: O(2^n)
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

# ✅ Мемоизация: O(n)
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]

# ✅ Табуляция: O(n)
def fib_tab(n):
    if n <= 1:
        return n
    dp = [0, 1]
    for i in range(2, n+1):
        dp.append(dp[i-1] + dp[i-2])
    return dp[n]
```

2. Задача о рюкзаке (Knapsack):

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0]*(capacity+1) for _ in range(n+1)]

    for i in range(1, n+1):
        for w in range(capacity+1):
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w], dp[i-1][w-weights[i-1]] + values[i-1])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][capacity]
```

---

6. Жадные алгоритмы (Greedy) 💰

Идея: На каждом шаге выбираем локально оптимальное решение в надежде, что оно приведет к глобальному оптимуму.

Пример: Размен монет (для канонических систем):

```
def coin_change(coins, amount):
    coins.sort(reverse=True)
    count = 0
    for coin in coins:
        count += amount // coin
        amount %= coin
    return count if amount == 0 else -1
```

Когда работает: Задача о рюкзаке (непрерывная), алгоритм Дейкстры, минимальное остовное дерево (Крускал, Прим).

Когда НЕ работает: Задача о рюкзаке (0/1), задача коммивояжера.

---

7. Разделяй и властвуй (Divide & Conquer) ⚔️

Идея: Разбиваем задачу на независимые подзадачи, решаем их рекурсивно, объединяем результаты.

Примеры:

- MergeSort, QuickSort
- Бинарный поиск
- Умножение больших чисел (алгоритм Карацубы)
- Поиск ближайшей пары точек

📊 Сводная таблица алгоритмов

Категория	Алгоритм	Сложность	Применение
Сортировка	QuickSort	O(n log n)	Универсальная
Сортировка	MergeSort	O(n log n)	Linked lists, внешняя
Сортировка	HeapSort	O(n log n)	Мало памяти
Поиск	Binary Search	O(log n)	Отсортированные массивы
Графы	BFS	O(V+E)	Кратчайший путь (невзвешенный)
Графы	DFS	O(V+E)	Обход, топологическая сортировка
Графы	Dijkstra	O((V+E)logV)	Кратчайший путь (взвешенный)
DP	Fibonacci	O(n)	Пример перекрывающихся подзадач
DP	Knapsack	O(nW)	Оптимизация ресурсов

🎯 Как выбрать алгоритм?

1. **Нужно отсортировать?** → QuickSort (универсально), MergeSort (linked list), HeapSort (мало памяти).
2. **Поиск в отсортированном?** → Binary Search O(log n).
3. **Кратчайший путь в графе?** → BFS (невзвешенный), Dijkstra (взвешенный без отрицательных весов).
4. **Задача с перекрывающимися подзадачами?** → Динамическое программирование.
5. **Можно решать локально оптимально?** → Жадный алгоритм.

> **Золотое правило:** Сначала поймите ограничения (время, память, размер данных), затем выберите алгоритм с подходящей сложностью.

💡 Практическое применение

1. **Базы данных:** Сортировка и слияние для JOIN, B-деревья для индексов.
2. **Поисковики:** PageRank (графы), индексация (хэш-таблицы, деревья).
3. **Сети:** Маршрутизация (Dijkstra, Bellman-Ford).
4. **Компрессия:** Huffman coding (жадный алгоритм).
5. **Криптография:** Быстрое возведение в степень (Divide & Conquer).
6. **Машинное обучение:** Градиентный спуск (оптимизация), кластеризация.

> **Запомните:** Знание алгоритмов не сделает вас программистом автоматически. Но оно даст вам инструменты для решения сложных задач эффективно. Практикуйтесь на LeetCode, Codeforces, HackerRank!